

CO2_AT1

18. Evaluate Quick Sort performance under adaptive pivot strategies that change dynamically during execution. Record execution time across multiple runs. Analyze how adaptability impacts performance consistency. Interpret results and justify dynamic optimization techniques.

PROGRAM

```
import random
import time

# -----
# Pivot Selection Strategies
# -----

def first_pivot(arr, low, high):
    return low

def random_pivot(arr, low, high):
    return random.randint(low, high)

def median_of_three(arr, low, high):
    mid = (low + high) // 2
    a = arr[low]
    b = arr[mid]
    c = arr[high]

    if (a <= b <= c) or (c <= b <= a):
        return mid
    elif (b <= a <= c) or (c <= a <= b):
        return low
    else:
        return high
```

```

# Adaptive Pivot Strategy
def adaptive_pivot(arr, low, high):
    size = high - low + 1

    if size < 20:
        return first_pivot(arr, low, high)
    elif size < 100:
        return random_pivot(arr, low, high)
    else:
        return median_of_three(arr, low, high)

# -----
# Partition Function
# -----
def partition(arr, low, high):
    pivot_index = adaptive_pivot(arr, low, high)
    arr[pivot_index], arr[high] = arr[high], arr[pivot_index]

    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# -----
# Quick Sort
# -----
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)

```

```

        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# -----
# Main Program
# -----
SIZE = 5000
RUNS = 5

print("Adaptive Quick Sort Performance\n")

times = []

for run in range(1, RUNS + 1):
    data = random.sample(range(1, 50000), SIZE)

    start = time.perf_counter()

    quick_sort(data, 0, len(data) - 1)

    end = time.perf_counter()

    elapsed = end - start
    times.append(elapsed)

    print(f"Run {run}: {elapsed:.6f} seconds")

average = sum(times) / RUNS

print("\nAverage Execution Time:", round(average, 6), "seconds")

```

Output

main.py

Share

Run

```
1 import random
2 import time
3
4 # -----
5 # Pivot Selection Strategies
6 # -----
7 def first_pivot(arr, low, high):
8     return low
9
10 def random_pivot(arr, low, high):
11     return random.randint(low, high)
12
13 def median_of_three(arr, low, high):
14     mid = (low + high) // 2
15     a = arr[low]
16     b = arr[mid]
17     c = arr[high]
18
19     if (a <= b <= c) or (c <= b <= a):
20         return mid
21     elif (b <= a <= c) or (c <= a <= b):
22         return low
23     else:
24         return high
25
```

Output

Adaptive Quick Sort Performance

Run 1: 0.007509 seconds

Run 2: 0.006915 seconds

Run 3: 0.008101 seconds

Run 4: 0.007474 seconds

Run 5: 0.008799 seconds

Average Execution Time: 0.007759 seconds

=== Code Execution Successful ===